

Beauty Books World

Live Server vs Local Environment Issue Analysis Report

Prepared for: Beauty Books World Customization (BuildStart Self-Hosted Platform)

Date: September 21, 2026

Document Type: Technical Root-Cause Analysis (RCA) & Solution Guide

1. Executive Summary

During the development and testing of the **Beauty Books World** custom WhatsApp sales agent on a local environment, the conversational flow, consultative sales steps, private branding validations, and order tracking functioned as expected. However, once deployed and executed on the **Live Server**, five critical functional discrepancies were encountered:

- System Prompt Sequence Failure:** AI did not follow the sequential 7-step consultative flow defined in the prompt.
- Private Branding Verification Failure:** Choosing "Yes" for Private-label Branding did not reflect "Need Verification" / "Need Manual Attention" on the chat.
- Max Discount Calculation Failure:** Configured maximum discount (20%) was not computed on orders.
- Payment Slip Number Hallucination:** The AI invented/guessed a random WhatsApp phone number for deposit slip verification instead of using the company's designated number (`0761309690`).
- Orders Tab Data Invisibility:** Completed orders did not appear in the dashboard Orders tab.

The Underlying Core Reason

In local development, custom database columns (`custom_fields`), dynamic settings (`custom_chat_flow` , `max_discount` , `payment_info`), and seeding scripts (`seed_beauty_books_complete.ts`) were executed directly on the developer's local Postgres instance. When deploying to the Live Server, the database was initialized using the static `db/01_schema.sql` and `02_seed.sql` , which lacked the custom tenant data, causing silent database insert rejections and prompt degradation.

2. Deep Dive: `custom_fields` and `custom_chat_flow`

To understand why these issues occurred, it is essential to review why `custom_fields` and `custom_chat_flow` were created and how they are architected.

2.1 What is `custom_fields` and Why Was It Created?

BuildStart is built on a **Multi-Tenant Shared Database Architecture** (as documented in `DATABASE_IMPLEMENTATION.md`).

- **The Problem:** The standard `orders` table only supports generic e-commerce attributes (`customer_name` , `customer_phone` , `customer_address` , `order_items` , `payment_method` , `total_amount` , `status`).
- **Client Specifics:** Beauty Books World has bespoke cosmetic manufacturing attributes that generic platforms do not have (Packaging format, MOQ rules, Private-Label Branding requirement, Discount percentages, Operational flags).
- **The Architecture Decision:** Rather than executing multiple destructive `ALTER TABLE` migrations on a shared multi-tenant database to add columns that only Beauty Books uses, a single **JSONB metadata column** named `custom_fields` was adopted.

```
{
  "purchase_type": "50g Standard Jar",
  "quantity": 12,
  "packaging_option": "Standard Jar",
  "branding_req": "Yes",
  "sample_request": "No",
  "discount_percentage": 20,
  "discount_amount": 4380,
  "quotation_status": "Sent",
  "payment_status": "Pending",
  "follow_up_status": "Manual Follow-Up Required",
  "manual_handoff_status": "Manual Follow-Up Required"
}
```

Why it failed on Live Server: The SQL column definition was never added to the base file `db/01_schema.sql` . When the live database was initialized, `orders` was created **without** `custom_fields` . Every order insert executed by the edge function crashed with a Postgres syntax error: `column "custom_fields" of relation "orders" does not exist`

2.2 What is `custom_chat_flow` and Why Was It Created?

Standard AI chatbots simply answer questions reactively. However, Beauty Books requires a **Consultative Sales Process** (12-step flow outlined in `prd_text.txt`).

- **The Problem:** Hardcoding business rules directly inside universal edge functions makes the codebase rigid and prevents business owners from updating sales rules.
- **The Solution:** The flow was codified into `beauty_books_faqs.json` and designed to be stored in the database under `public.settings` where `key = 'custom_chat_flow'` .

Runtime Integration: In `supabase/functions/ai-chat/index.ts` :

```
const customChatFlow = settings.find(s => s.key === "custom_chat_flow")?.value?.text || "";

const systemPrompt = `...
${customChatFlow ? `
--- MANDATORY STEP-BY-STEP CONSULTATIVE SALES FLOW ---
You MUST follow this exact sequence based on the conversation history. DO NOT skip or merge
steps out of order:
1. GREETING -> STEP 1: Welcome message & 7 services
2. SERVICE SELECTION -> Routing
3. PRODUCT & FORMULA SELECTION -> Benefits & Catalog
4. FORMAT & QUANTITY -> MOQ Validation & Branding Question
5. AFTER BRANDING ("Yes"/"No") -> STEP 8: Order Summary with Discounts
6. CONFIRMATION -> Delivery Details
7. PAYMENT & ORDER_JSON -> Bank info + 0761309690 + <ORDER_JSON>
` : ` - If a customer wants to order, guide them through collecting: name, phone...`}`;
```

Why it failed on Live Server: On a fresh Postgres deployment, the database trigger `handle_new_user_settings()` only auto-creates `welcome_message` , `payment_info` , and `auto_responses` . It **does not** create `custom_chat_flow` . In local, running `seed_beauty_books_complete.ts` populated this row. On the Live Server, this seed script had not been executed, causing `customChatFlow` to be empty, bypassing the 7-step sequence entirely!

3. Comprehensive Breakdown of the 5 Issues

#	Issue Description	Why it Worked in Local	Why it Failed on Live Server
1	Sequence rules not followed by AI	Local DB had <code>custom_chat_flow</code> populated in <code>settings</code> table via seed script.	Live DB <code>settings</code> lacked <code>custom_chat_flow</code> . Prompt fell back to generic chatbot instructions.
2	Private branding "Yes" did not trigger "Need Verification"	In local, orders saved successfully, which triggered the <code>chat_takeovers</code> upsert.	1) Order insert crashed on Live, so <code>chat_takeovers</code> was never upserted. 2) Takeover code is only triggered upon order completion.
3	Max discount not calculated (0%)	Local settings had <code>max_discount: { percentage: 20 }</code> .	Live DB lacked <code>max_discount</code> row in <code>settings</code> . Defaulted to <code>0%</code> .

#	Issue Description (Cont.)	Why it Worked in Local	Why it Failed on Live Server
4	Payment slip number hallucinated by AI	Local prompt had <code>custom_chat_flow</code> containing explicit instructions with <code>0761309690</code> .	Live DB <code>payment_info</code> was empty, and missing <code>custom_chat_flow</code> stripped instructions, causing hallucinations.
5	Orders tab empty after order completion	Local database had column <code>custom_fields</code> created manually on <code>public.orders</code> .	Live DB was created from <code>01_schema.sql</code> which lacks <code>custom_fields</code> . Insert crashed. Also, token truncation hid the <code><ORDER_JSON></code> .

4. Edge-Case Technical Factors

4.1 Token Truncation (`MAX_TOKENS = 500`)

In `supabase/functions/ai-chat/index.ts`:

```
const MAX_TOKENS = 500;
```

When completing an order, the AI must output bank details, deposit instructions, multilingual closings, and a large JSON payload (`<ORDER_JSON>`). At 500 tokens, the response frequently truncates before generating `</ORDER_JSON>`. When truncated, regex parsing fails, and the post-processor strips the incomplete tag, leaving no order record.

4.2 Mid-Conversation Branding Takeover

In `ai-chat/index.ts`, the trigger for `chat_takeovers` is placed inside the order completion block:

```
if (isBrandingRequired || computedManualHandoff === "Manual Follow-Up Required") {
  await supabase.from("chat_takeovers").upsert({ ... });
}
```

If a customer answers "Yes" to branding at Step 4/5, but has not yet confirmed the order at Step 11, the dashboard will not display "Need Manual Attention" until the order process finishes.

5. Strategic Recommendations & Action Plan

To establish absolute stability across local, staging, and production environments, the following two-phase solution is recommended:

Phase 1: Server-Side Fixes (Can be executed immediately)

1. Add `custom_fields` Column to Live Postgres:

Run this query directly in Supabase SQL Editor on the live server:

```
ALTER TABLE public.orders
ADD COLUMN IF NOT EXISTS custom_fields JSONB DEFAULT '{} '::jsonb;
```

2. Seed Beauty Books Settings & Flow for Live User:

Run the seed script on the live server using the live account's UUID:

```
deno run --allow-net --allow-read --allow-env seed_beauty_books_complete.ts
<YOUR_LIVE_USER_UUID>
```

(Alternatively, SQL inserts can be run directly to populate `max_discount` and `payment_info`.)

3. Update Dashboard Settings:

- In Dashboard, go to **Settings -> Discount Settings**, enter **20%**, and click Save.
- In Dashboard, go to **Settings -> Payment Settings**, verify bank details and enter **0761309690** as the payment verification number.

Phase 2: Codebase Fixes (Permanent Codebase Integrity)

- **Increase Max Tokens:** In `ai-chat/index.ts`, change `MAX_TOKENS = 500;` to `1200;` to guarantee the AI never truncates the order payload.
- **Synchronize Base Schema:** Add `custom_fields jsonb DEFAULT '{} '::jsonb` to `public.orders` in `db/01_schema.sql`.
- **Immediate Chat Takeover:** Add immediate detection in `ai-chat/index.ts` so a "Yes" to Private Branding triggers an immediate `chat_takeovers` upsert mid-chat.

6. End-to-End Verification Checklist

- ☐ Send "Hi" -> Verify welcome message and 7 service options appear.
- ☐ Choose "1" -> Verify Whitening Cream / Formula options appear.
- ☐ Reply "Formula 07" -> Verify benefits are stated and formats/prices are listed.
- ☐ Reply "12 pcs" -> Verify MOQ is accepted, AI explicitly asks: "Would you like to add our Private-Label Branding service for LKR 5,000?".
- ☐ Reply "Yes" -> Verify Order summary calculates 20% discount and branding fee.
- ☐ Reply "Confirmed" -> Verify delivery details are requested.
- ☐ Send address -> Verify bank details, `0761309690` is stated, order appears in Dashboard, and "Need Manual Attention" badge shows up.